SliceTune

Készítette: Szemán Gergő és Máder-Piros Botond

Konzulensek: Szilasi István és Dorogházi Donát

1. Projekt áttekintés 4
1.1 Technológiai stack 4
1.1.1 Backend 4
1.1.2 Frontend 4
1.1.3 Adatbázis 4
1.1.4 Autentikáció 4
1.1.5 Fájlfeltöltés 5
1.1.6 Külső API 5
1.2 Projektstruktúra 5
2. Architektúra 6
2.1 ERD diagram 6
2.1.1. Felhasználók 6
users 6
2.1.1 Zenék és kapcsolódó adatok 7
musics 7
artists 7
albums 7
2.1.3. Lejátszási listák 8
playlists 8
playlist_musics 8
2.1.4. Fájlok 8
music_files 8
image_files 8
2.1.5. Beállítások és egyéb funkciók 9
user_settings 9
reports 9
2.1.6. Kapcsolatok összefoglalása 9
2.2 Normálformák 10
2.2.1. Első normálforma (1NF) 10
2.2.2. Második normálforma (2NF) 10
2.2.3. Harmadik normálforma (3NF) 10
2.2.4. Összegzés 10
2.3.Fronted zene hozzáadás/módosítás 11
3. Backend dokumentáció 12
3.1 Szerver konfiguráció (app.ts) 12
3.2 Adatbázis-konfiguráció (config/config.ts) 12
3.3 Middleware 12
3.3.1 JWT hitelesítés (middleware/auth.ts) 12
3.3.2 Fájlfeltöltés (middleware/upload.ts) 13
3.4 Segédeszközök (utils/fileUtils.ts) 13
3.5 API végpontok 13
3.5.1 Zenék (/musics) 13
3.5.2 Felhasználók (/users) 13
3.5.3 Lejátszási listák (/playlists) 14
3.5.4 Albumok (/albums) 14
3.5.5 Előadók (/artists) 15

3.5.6 Fájlok (/files)....15
3.5.7 Keresés (/search)....15
3.5.8 Beállítások (/settings)....16
4. Adatmodellek.... 18
5. Frontend dokumentáció.... 19
5.1 Fő modulok.... 19
5.1.1 auth.js – Hitelesítés kezelése....19
5.1.2 data.js – API kommunikáció....19
5.1.3 playerLogic.js – Zenelejátszó logika.... 19
5.2 Komponensek.... 19
6. Biztonsági megjegyzések.... 20
6.1 JWT hitelesítés....20
6.2 Fájlbiztonság....20
6.3 Jogosultságkezelés....20
6.4 Ismert fejlesztési lehetőségek....20
7. Telepítési útmutató.... 21
7.1 Követelmények....21
7.2 Backend indítása....21
7.3 Frontend indítása....21
7.4 Uploads mappa....21

1. Projekt áttekintés

A SliceTune egy teljes értékű zenei streaming webalkalmazás, amely lehetővé teszi a felhasználók számára zenefájlok feltöltését, hallgatását, lejátszási listák kezelését, valamint zenék keresését.

1.1 Technológiai stack

Réteg	Technológia
Backend	Node.js, Express.js, TypeScript
Frontend	React (JSX), Vite
Adatbázis	MySQL (mysql2/promise)
Autentikáció	JWT (jsonwebtoken)
Fájlfeltöltés	Multer (multipart/form-data)
Külső API	iTunes Search API (zenekeresés)

1.1.1 Backend

- Node.js - Egy nyílt forráskódú futtatókörnyezet, amely lehetővé teszi, hogy a JavaScriptet ne csak böngészőben, hanem szerver oldalon is használható legyen. Gyors és eseményvezérelt működése miatt gyakran alkalmazzák webalkalmazások és API-k fejlesztésére.
- Express.js - Egy minimalista és rugalmas webes keretrendszer a Node.js-hez, amely megkönnyíti szerverek és API-k gyors fejlesztését.
- TypeScript - A JavaScript egy kibővített változata, amely statikus típuskezelést ad hozzá, így könnyebb vele nagyobb, átláthatóbb alkalmazásokat fejleszteni.

1.1.2 Frontend

- React (JSX) - Egy UI-könyvtár, amelyet főként felhasználói felületek építésére használnak. A JSX egy speciális szintaxis benne, amely lehetővé teszi, hogy HTML-szerű kódot írj közvetlenül JavaScriptben.
- Vite - Egy modern frontend build eszköz, amely gyors fejlesztői szervert és villámgyors újratöltést biztosít, főként olyan keretrendszerekhez, mint a React.

1.1.3 Adatbázis

- MySQL - Egy nyílt forráskódú relációs adatbázis-kezelő rendszer, amely strukturált adatok tárolására és lekérdezésére szolgál, és gyakran használják webalkalmazások háttérrendszereként.

1.1.4 Autentikáció

- JSON Web Token - Egy szabványos, tömör token formátum, amelyet főként hitelesítésre és jogosultságkezelésre használnak webalkalmazásokban. A token digitálisan aláírt, így biztonságosan továbbítható a kliens és a szerver között.

1.1.5 Fájlfeltöltés

- Multer - Egy middleware a Node.js-hez (gyakran Express.js-szel használva), amely fájlfeltöltések kezelésére szolgál, például űrlapokból érkező képek vagy dokumentumok feldolgozására.

1.1.6 Külső API

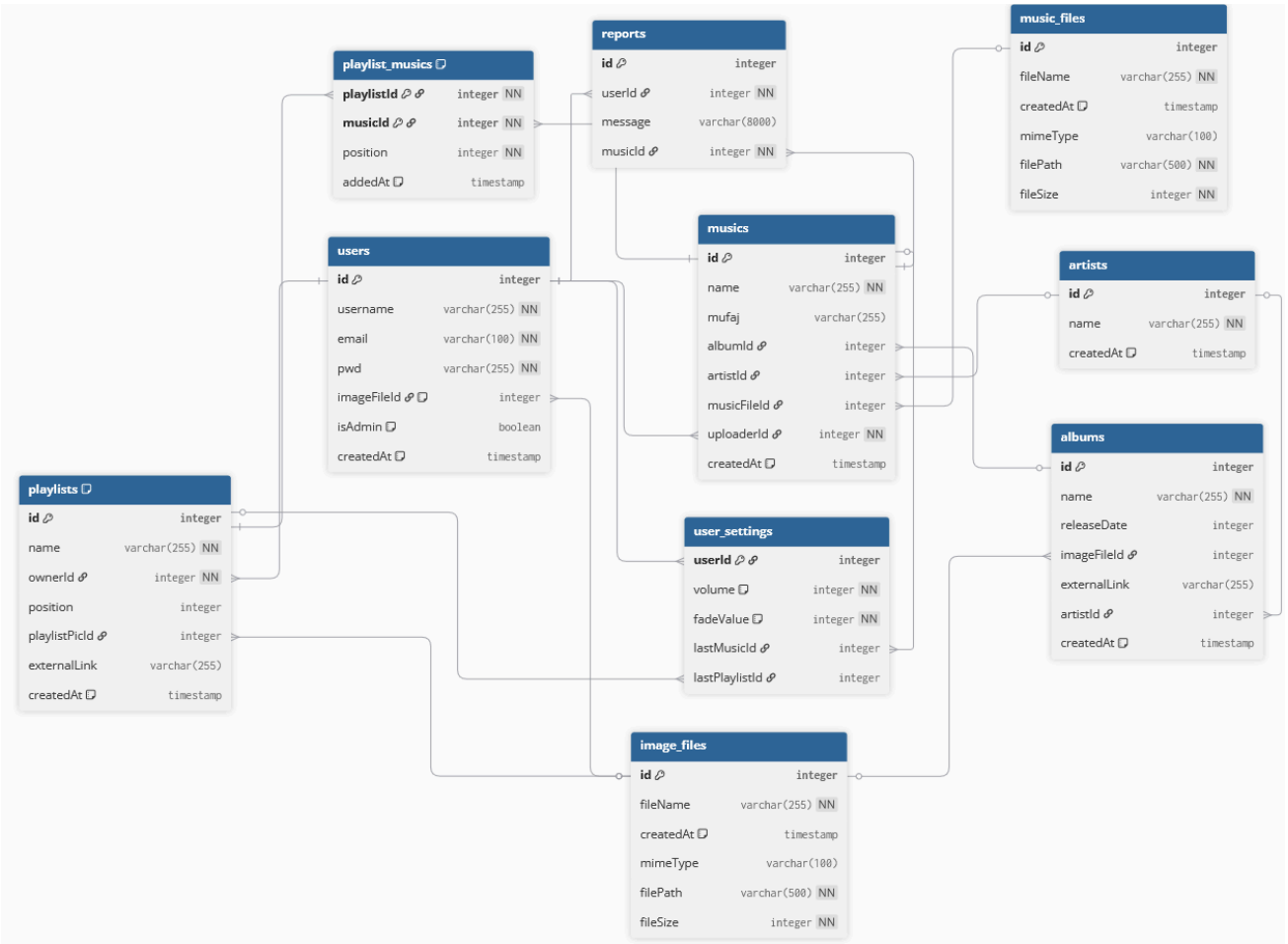
- iTunes Search API - Egy külső API, amit a projekt a zenékről való információjához használ a felhasználó dolgának megkönnyítésére. A weboldal e nélkül is használható.

1.2 Projektstruktúra

```
Vizsgaremek/
├── backend/          # Node.js + Express API szerver
│   ├── src/
│   │   ├── app.ts             # Belépési pont, Express konfiguráció
│   │   ├── config/            # Adatbázis-kapcsolat és környezeti változók
│   │   ├── classes/           # Adatmodellek (User, Music, Playlist, ...)
│   │   ├── controllers/       # Üzleti logika és HTTP-kezelők
│   │   ├── middleware/        # JWT-hitelesítés, fájlfeltöltés
│   │   ├── routes/            # Express útvonalak
│   │   └── utils/             # Segédeszközök (fájltörlés)
│   └── uploads/               # Feltöltött zene- és képfájlok
└── frontend/         # React + Vite kliens
    └── src/
        ├── components/        # UI komponensek
        ├── styles/            # CSS stíluslapok
        ├── auth.js            # JWT-kezelés
        ├── data.js            # API-hívások
        └── playerLogic.js     # Zenelejátszó logika
```

2. Architektúra

2.1 ERD diagram



2.1.1. Felhasználók

`users`

A rendszer felhasználóit tárolja.

- id: egyedi azonosító
- username, email, pwd: bejelentkezési adatok
- imageFileId: profilkép hivatkozás
- isAdmin: admin jogosultság
- createdAt: létrehozás ideje

Kapcsolatok:

- egy felhasználónak több playlistje lehet
- egy felhasználó tölthet fel zenéket
- tartozik hozzá egy beállítás rekord

2.1.1 Zenék és kapcsolódó adatok

`musics`

A központi tábla, minden zene itt van.

- id, name
- albumId: album
- artistId: előadó
- musicFileId: hangfájl
- uploaderId: feltöltő
- createdAt

Kapcsolatok:

- egy zene egy albumhoz és egy előadóhoz tartozik
- több playlistben is szerepelhet

`artists`

Előadók adatai:

- id, name, createdAt

Kapcsolatok:

- egy előadóhoz több album és zene tartozik

`albums`

Albumok:

- id, name
- releaseDate
- imageFileId: borítókép
- artistId: előadó
- externalLink

Kapcsolatok:

- egy albumhoz több zene tartozik

2.1.3. Lejátszási listák

`playlists`

Playlist-ek:

- id, name
- ownerId: tulajdonos
- position: sorrend
- playlistPicId: kép
- externalLink

`playlist_musics`

Kapcsolótábla (many-to-many kapcsolat):

- playlistId
- musicId
- position: hely a listában
- addedAt

Ez biztosítja, hogy:

- egy playlist több zenét tartalmazhat
- egy zene több playlistben is benne lehet

2.1.4. Fájlok

`music_files`

Hangfájlok:

- fileName, filePath
- mimeType, fileSize
- createdAt

`image_files`

Képfájlok:

- fileName, filePath
- mimeType, fileSize
- createdAt

Megjegyzés: a fájlok külön táblákban vannak, így a rendszer jobban skálázható és rendezettebb.

2.1.5. Beállítások és egyéb funkciók

`user_settings`

Felhasználói beállítások:

- userId
- volume: hangerő
- fadeValue: átmenet
- lastMusicId: utoljára hallgatott zene
- lastPlaylistId

`reports`

Jelentések (pl. problémás tartalom):

- userId: ki jelentette
- musicId: melyik zenét
- message: leírás

2.1.6. Kapcsolatok összefoglalása

- users → playlists: egy-a-többhöz
- users → musics (feltöltő): egy-a-többhöz
- artists → albums → musics: hierarchikus kapcsolat
- musics ↔ playlists: több-a-többhöz (playlist_musics táblával)
- fájlok külön táblákból vannak hivatkozva
- users → reports → musics: jelentési rendszer

2.2 Normálformák

2.2.1. Első normálforma (1NF)

Definíció:

- Minden mező **atomi** (nem tartalmaz listát vagy tömböt)
- Nincsenek ismétlődő csoportok
- Minden rekord egyértelműen azonosítható (primer kulcs)

Példa hibára: egy mezőben több zene ID vesszővel elválasztva

2.2.2. Második normálforma (2NF)

Definíció:

- Teljesíti az 1NF-et
- Minden nem kulcs mező **teljes mértékben függ a teljes primer kulcstól**
- Nincs részleges függés (főleg összetett kulcsnál fontos)

Példa hibára:
Ha egy összetett kulcsú táblában egy mező csak az egyik kulcstagtól függ

2.2.3. Harmadik normálforma (3NF)

Definíció:

- Teljesíti a 2NF-et
- Nincs **tranzitív függés**

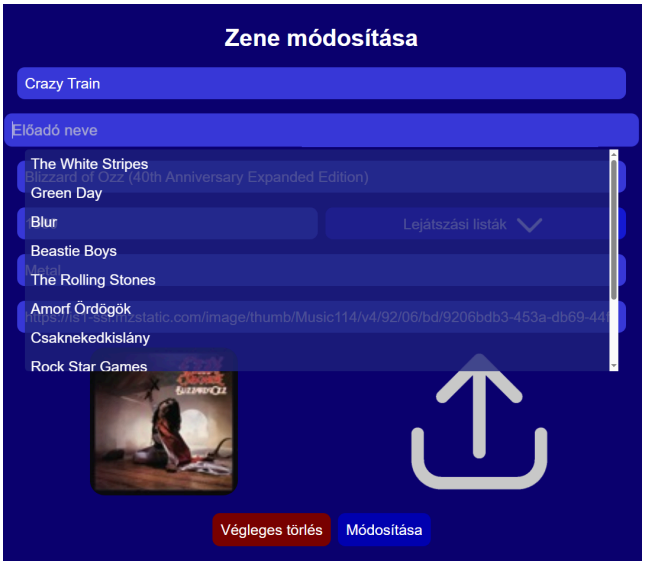
Példa hibára:

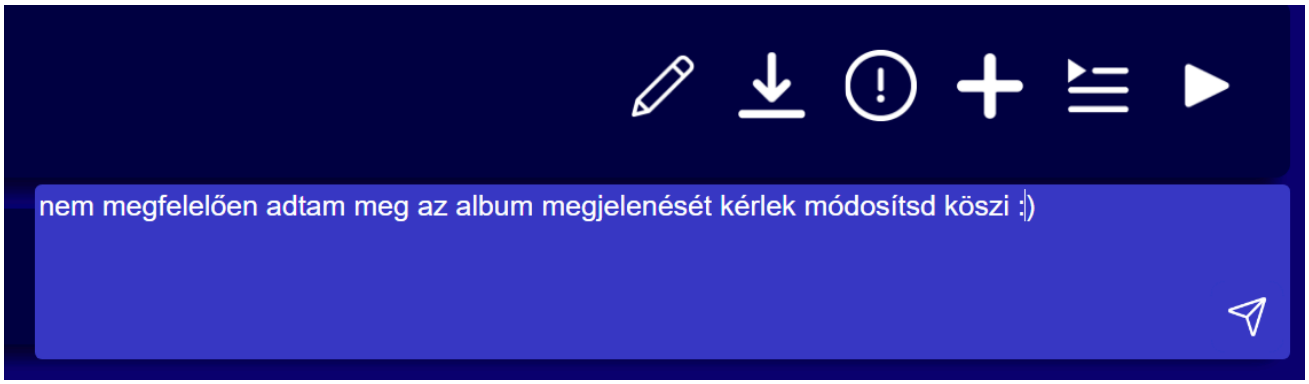
- music táblában lenne artistName is, miközben az artistId már megvan

2.2.4. Összegzés

Az adatbázis a három normálformának megfelel.

2.3 Frontend zene hozzáadás/módosítás





A zenéket létre lehet hozni saját kezűleg és ha úgy tartja kedved tudod módosítani az atribútumait viszont fontos hozzátenni hogy az album és az előadót csak úgy tudja módosítani a felhasználó hogy újat hoz létre mert ha esetleg valaki más is használatba vette az általa létrehozott albumot akkor ne tudja megváltoztatni csak úgy ha más is használná esetleg de ha valami probléma van az albummal vagy az előadóval akkor a reportok küldésével képes szólni az adminnak hogy mi az amit elírt elrontott vagy meg kéne változtatni az albumon és ha ténylegesen olyan hiba áll fenn akkor az admin ezt képes módosítani.

Annak érdekében hogy elkerüljük az ilyesfajta hibákat és hogy egyszerűsítsük a zene létrehozását a felhasználónak felhasználjuk az ITunes ingyenes apiját amivel le tudjuk kérni azokat az adatokat amiket kér a formunk és annak érdekében hogy biztos a jó zenét adja hozzá az ITunes prevue funkcióját használva bele tud hallgatni a zenébe amit hozzá akar adni hogy biztos jót adjon hozzá.

3. Backend dokumentáció

3.1 Szerver konfiguráció (app.ts)

Az alkalmazás belépési pontja. Beállítja az Express middleware-eket, CORS-t, és regisztrálja az összes útvonacsoportot.

Fontos beállítások:

- Port: PORT env változó, alapértelmezetten 3000
- CORS: csak a http://localhost:5173 eredetről fogad el kéréseket
- Statikus fájlszolgáltatás: /uploads mappa
- iTunes proxy végpont: GET /api/search – a frontend oldalon proxyzza az iTunes API-t

3.2 Adatbázis-konfiguráció (config/config.ts)

MySQL kapcsolatot hoz létre .env fájl alapján. A konfigurációs objektum a következő értékeket tartalmazza:

- connection – mysql2 adatbázis-kapcsolat
- jwtSecret – JWT aláíráshoz használt titkos kulcs
- baseDir – projekt gyökérkönyvtár abszolút elérési útja
- musicUploadDir – zenefájlok feltöltési könyvtára
- imageUploadDir – képfájlok feltöltési könyvtára

Szükséges .env változók:

```
DB_HOST=
DB_USER=
DB_PASSWORD=
DATABASE=slicetune
JWT_SECRET=
PORT=
MUSIC_UPLOAD_DIR=/uploads/musics/
IMAGE_UPLOAD_DIR=/uploads/images/
```

3.3 Middleware

3.3.1 JWT hitelesítés (middleware/auth.ts)

A verifyToken middleware ellenőrzi a JWT tokent. A tokent a következő helyekről olvassa be (prioritás sorrendben):

- req.body.token – kérés törzse
- req.query.token – URL lekérdezési paraméter
- req.headers["x-access-token"] – HTTP fejléc

Hiba esetén 403-as (nincs token) vagy 401-es (érvénytelen token) státuszt ad vissza.

3.3.2 Fájlfeltöltés (middleware/upload.ts)

Multer alapú feltöltő middleware két típussal:

- uploadMusicMiddleware – zenefájlok, max. 100 MB, a musicUploadDir könyvtárba
- uploadImageMiddleware – képfájlok, max. 20 MB, az imageUploadDir könyvtárba

Mindkét middleware Promise-ként is használható (util.promisify).

3.4 Segédeszközök (utils/fileUtils.ts)

safeUnlink(filePath: string) – Biztonságos fájltörlő függvény. Ellenőrzi, hogy a törlendő fájl az uploads könyvtáron belül van-e (directory traversal védelem), majd törli azt. Ha a fájl már nem létezik (ENOENT), nem dob hibát.

3.5 API végpontok

3.5.1 Zenék (/musics)

Metódus	Végpont	Auth	Leírás
GET	/musics	Nem	Összes zene listázása
GET	/musics/:id	Nem	Zene lekérése ID alapján (előadó, album, URL-lel együtt)
GET	/musics/byplaylistid/:id	Nem	Lejátszási lista zenéinek lekérése
GET	/musics/byuserid/:id	Nem	Felhasználó feltöltött zenéi
GET	/musics/album/:id	Nem	Album zenéi
POST	/musics	Igen	Új zene létrehozása
PUT	/musics/:id	Igen	Zene adatainak módosítása
DELETE	/musics/:id	Igen	Zene törlése (zenefájllal együtt, ha nincs más hivatkozás)

getMusicById megjegyzés:

Ez a végpont több táblát is lekérdez (musics, artists, albums, image_files, music_files) és egy összetett JSON objektumot ad vissza, amely tartalmazza az előadó nevét, az album nevét, a borítókép URL-jét és a zenefájl elérési útját.

3.5.2 Felhasználók (/users)

Metódus	Végpont	Auth	Leírás
POST	/users/login	Nem	Bejelentkezés, JWT token visszaadása
POST	/users/signup	Nem	Regisztráció
GET	/users	Igen	Összes felhasználó (admin)
GET	/users/getuser/:id	Igen	Felhasználói adatok (csak saját, vagy admin)

Metódus	Végpont	Auth	Leírás
PUT	/users/:id	Igen	Felhasználói adatok módosítása
DELETE	/users/:id	Igen	Felhasználó törlése (profilképpel együtt)
POST	/users/passcheck	Igen	Jelszó ellenőrzése
POST	/users/report	Igen	Bejelentés létrehozása
DELETE	/users/delreport/:id	Igen	Bejelentés törlése
GET	/users/reportByUserId/:id	Nem	Felhasználó bejelentései
GET	/users/reportByMusicId/:id	Nem	Zene bejelentései

3.5.3 Lejátszási listák (/playlists)

Metódus	Végpont	Auth	Leírás
GET	/playlists	Nem	Összes lejátszási lista
GET	/playlists/:id	Nem	Lista lekérése ID alapján
GET	/playlists/byuserid/:id	Igen	Felhasználó listái
POST	/playlists	Igen	Új lista létrehozása
POST	/playlists/addMusic	Igen	Zene hozzáadása listához
POST	/playlists/getPlaylistsByIds	Igen	Listázza, melyik listában van egy adott zene
DELETE	/playlists/removeMusic	Igen	Zene eltávolítása listából
DELETE	/playlists/:id	Igen	Lista törlése (csak a tulajdonos által)
PUT	/playlists/:id	Igen	Lista adatainak módosítása
PUT	/playlists/updateplaylistpos/...	Igen	Zene pozíciójának módosítása a listában

3.5.4 Albumok (/albums)

Metódus	Végpont	Auth	Leírás
GET	/albums	Nem	Összes album
GET	/albums/:id	Nem	Album lekérése ID alapján
GET	/albums/artist/:id	Nem	Előadó albumjai

Metódus	Végpont	Auth	Leírás
POST	/albums	Igen	Új album létrehozása
PUT	/albums/:id	Igen	Album módosítása
DELETE	/albums/:id	Igen	Album törlése

3.5.5 Előadók (/artists)

Metódus	Végpont	Auth	Leírás
GET	/artists	Nem	Összes előadó
GET	/artists/:id	Nem	Előadó lekérése ID alapján
POST	/artists	Igen	Új előadó létrehozása
PUT	/artists/:id	Igen	Előadó módosítása
DELETE	/artists/:id	Igen	Előadó törlése

3.5.6 Fájlok (/files)

Metódus	Végpont	Auth	Leírás
GET	/files/music	Nem	Zenefájlok listája
GET	/files/music/:id	Nem	Zenefájl lekérése ID alapján
GET	/files/image/:id	Nem	Képfájl URL-jének lekérése
GET	/files/images	Nem	Képfájlok listája
POST	/files/music	Igen	Zenefájl feltöltése
POST	/files/image	Igen	Képfájl feltöltése
GET	/files/download/music/:id	Nem	Zenefájl letöltése
GET	/files/download/image/:id	Nem	Képfájl letöltése
DELETE	/files/music/:id	Igen	Zenefájl törlése (lemezről is)
DELETE	/files/image/:id	Igen	Képfájl törlése (lemezről is)

3.5.7 Keresés (/search)

Metódus	Végpont	Auth	Leírás
GET	/search/musics?name=...	Nem	Zenék keresése cím alapján

Metódus	Végpont	Auth	Leírás
GET	/search/musicsbyartist?name=...	Nem	Zenék keresése előadó neve alapján
GET	/search/musicsbyalbum?name=...	Nem	Zenék keresése album neve alapján
GET	/search/musicsbyusername?name=...	Nem	Zenék keresése feltöltő neve alapján
GET	/search/playlists?name=...	Nem	Lejátszási listák keresése név alapján
GET	/search/playlistsbyusername?name=...	Nem	Listák keresése felhasználónév alapján
GET	/search/users?username=...	Nem	Felhasználók keresése
GET	/search/reports?message=...	Nem	Bejelentések keresése
GET	/search/musicinplaylist?...	Nem	Zene keresése listán belül
GET	/search/musicinalbum?...	Nem	Zene keresése albumon belül
GET	/search/musicbyartist?...	Nem	Zene keresése előadón belül
GET	/search/albumbyartist?...	Nem	Album keresése előadón belül

3.5.8 Beállítások (/settings)

Metódus	Végpont	Auth	Leírás
GET	/settings	Nem	Összes felhasználói beállítás
GET	/settings/:id	Nem	Felhasználó beállításai
PUT	/settings/:id	Igen	Beállítások frissítése (volume, fadeValue, lastMusicId, lastPlaylistId)
PUT	/settings/removeplaylist/:id	Igen	lastPlaylistId törlése

3.6 Hibakódok

3.6.1 2xx

200 - OK
201 - Created
204 - No Content

3.6.1 4xx

400 - Bad Request
401 - Unauthorized
403 - Forbidden
404 - Not Found

4. Adatmodellek

A backend classes/ könyvtárában TypeScript osztályok definiálják az adatmodelleket.

Osztály	Főbb mezők
`Music`	id, name, mufaj (műfaj), albumId, artistId, musicFileId, uploaderId
`Album`	id, name, releaseDate, externalLink, artistId, imageFileId
`Artist`	id, name
`Playlist`	id, name, creatorId (ownerId), playlistPicId, externalLink, position
`PlaylistMusics`	playlistId, musicId, position
`Users`	id, username, email, pwd, imageFileId, isAdmin
`Settings`	userId, volume, fadeValue, lastMusicId, lastPlaylistId
`ImageFile`	id, fileName, mimeType, filePath, uploadedBy
`MusicFile`	id, fileName, mimeType, filePath

5. Frontend dokumentáció

5.1 Fő modulok

5.1.1 auth.js – Hitelesítés kezelése

JWT token mentése és olvasása (localStorage). Funkciókat biztosít a bejelentkezett felhasználó azonosítójának és tokenének lekéréséhez.

5.1.2 data.js – API kommunikáció

Centralizált helyen tartja az összes backend API hívást. Fetch alapú függvények a zene, felhasználó, lejátszási lista, album, előadó és fájl műveletek elvégzéséhez.

5.1.3 playerLogic.js – Zenelejátszó logika

A zenelejátszóhoz tartozó üzleti logika: következő/előző zene, véletlenszerű lejátszás, ismétlési módok kezelése, hangerő.

5.2 Komponensek

Komponens	Leírás
`MainPage.jsx`	Főoldal – a felhasználó lejátszási listáit és zenéit jeleníti meg
`player.jsx`	Zenelejátszó sáv – lejátszás, következő/előző, hangerő, haladásbár
`playlist.jsx`	Lejátszási lista oldalsáv – saját listák megjelenítése
`playlist-show.jsx`	Egy adott lejátszási lista tartalma
`playlist-row.jsx`	Egy lejátszási lista egy sorának megjelenítése
`kereses.jsx`	Keresési felület – cím, előadó, album, felhasználó szerinti szűrés
`loginOrSignin.jsx`	Bejelentkezési / regisztrációs űrlap
`createOrEditMusic.jsx`	Zene létrehozása vagy szerkesztése
`createOrEditPlaylist.jsx`	Lejátszási lista létrehozása / szerkesztése
`adminMusicEdit.jsx`	Admin: zene szerkesztése (előadó, album hozzárendelés)
`adminPlaylistEdit.jsx`	Admin: lejátszási lista szerkesztése
`setting.jsx`	Felhasználói beállítások oldal
`top-content.jsx`	Felső navigációs sáv
`report-row.jsx`	Egy bejelentési sor megjelenítése
`ErrorNotFound.jsx`	404-es hibaoldal

6. Biztonsági megjegyzések

6.1 JWT hitelesítés

- A tokenek 2 óráig érvényesek
- A védett végpontok mindegyike verifyToken middleware-t igényel
- A token tartalmazza a felhasználó ID-ját és e-mail-jét

6.2 Fájlbiztonság

- safeUnlink: ellenőrzi, hogy a törlendő fájl az uploads könyvtáron belül van-e (path traversal védelem)
- Fájlméret korlátozás: zenék max. 100 MB, képek max. 20 MB

6.3 Jogosultságkezelés

- getUserById: csak saját adat, vagy admin férhet hozzá
- deletePlaylist: csak a tulajdonos törölheti a saját listáját
- Jelszócsere: pwd_encrypt MySQL függvény titkosítja a jelszavakat*

6.4 Ismert fejlesztési lehetőségek

- updatePlaylistPosition: a függvény jelenleg nem await-eli megfelelően a lekérdezéseket (hiányzó await)
- Egyes végpontokon a Cache-Control fejléc hiányos – célszerű egységesíteni
- Az ertek / ertekek változónevek magyar megnevezéssel szerepelnek – refaktorálás javasolt
- A searchController nagy mértékben ismétlő kódot tartalmaz – közös segédfüggvény kiemelése ajánlott

7. Telepítési útmutató

7.1 Követelmények

- Node.js 18+
- MySQL 8+
- npm

7.2 Backend indítása

```
cd backend
npm install
# .env fájl létrehozása (lásd 2.2 fejezet)
# MySQL adatbázis importálása:
mysql -u root -p slicetune < slicetune.sql
npm run dev       # fejlesztői mód
npm run build     # production build
```

7.3 Frontend indítása

```
cd frontend
npm install
npm run dev       # Vite fejlesztői szerver (http://localhost:5173)
npm run build     # production build
```

7.4 Uploads mappa

A feltöltött fájlok a backend/uploads/ könyvtárban tárolódnak:

```
backend/uploads/musics/    # zenefájlok (.mp3, .wav, stb.)
backend/uploads/images/    # képfájlok (.jpg, .png, stb.)
```